

KAGGLE TITANIC · PRACTICAL SESSION

From raw data to your first model

Data cleaning, feature engineering, training, evaluation, and a valid Kaggle submission.

CLEAN

ENGINEER

TRAIN

What you will build today

1

Clean safely

Handle missing values without learning from validation or test data.

2

Create features

Turn family, name, and cabin information into useful signals.

3

Train a baseline

Build a reproducible Logistic Regression pipeline.

4

Submit correctly

Generate a two-column CSV with 418 predictions.

End result: code that treats training, validation, and test data consistently.

The modeling journey

1

Audit

Confirm target, types, missingness, and suspicious values.

2

Split

Reserve validation data before learning cleaning rules.

3

Prepare

Impute, encode, scale, and create features consistently.

4

Train

Fit the baseline only on the training portion.

5

Evaluate

Measure unseen validation predictions and inspect errors.

After evaluation: fit the chosen pipeline on all labeled rows, predict the test set, and create `submission.csv`.

Split before the data teaches you

#1

Prevent data leakage

Validation data must behave like **future, unseen passengers**.

- 1 Split labeled rows into **training** and **validation**.
- 2 Learn medians, categories, and model parameters from **training only**.
- 3 Apply those learned rules to validation — never refit on it.
- 4 Use a **Pipeline** so scikit-learn enforces the order.

Leakage: information unavailable at prediction time quietly enters training and produces an unrealistically good score.

Cleaning is decision-making

A

Missing values

Why are they missing? Can we impute, label, or safely remove them?

B

Types & labels

Are numbers truly numeric? Are categories spelled consistently?

C

Duplicates

Is a repeated row an error, or can two records legitimately match?

D

Outliers

Is an extreme value impossible, incorrect, or simply rare?

A cleaning choice should have a **reason**, a **method**, and a **check**.

Our decisions for Titanic

Column	Issue	Decision	Why?
Age	Many missing values	Median imputation	Robust to extreme ages
Fare	One test value missing	Median imputation	Keeps every test passenger
Embarked	A few values missing	Most frequent category	Simple categorical baseline
Cabin	Mostly missing	Create Deck; missing → U	Preserves “unknown” as information
Sex / Pclass	Categorical meaning	One-hot encoding	Creates model-readable indicators

Important: the imputer learns its median or mode during `fit()` using training rows only.

Turn raw columns into stronger clues

FAMILY **FamilySize**

SibSp + Parch + 1

How many relatives traveled together?

CONTEXT **IsAlone**

FamilySize == 1

Was the passenger traveling alone?

NAME **Title**

Mr, Mrs, Miss, Master...

A compact clue about age and social role.

CABIN **Deck**

First cabin letter

Approximate location; U means unknown.

Good feature engineering combines domain meaning with information available for both train and test passengers.

Build features once, reuse twice

```
def add_features(df):  
    out = df.copy()  
  
    out["FamilySize"] = out["SibSp"] + out["Parch"] + 1  
    out["IsAlone"] = (out["FamilySize"] == 1).astype(int)  
  
    out["Title"] = out["Name"].str.extract(  
        r",\s*([^.]*)\.", expand=False  
    ).str.strip()  
    common = ["Mr", "Mrs", "Miss", "Master"]  
    out["Title"] = out["Title"].where(out["Title"].isin(common), "Rare")  
  
    out["Deck"] = out["Cabin"].str[0].fillna("U")  
    return out
```

copy()

Protects the original DataFrame from accidental changes.

astype(int)

Turns True/False into model-friendly 1/0.

str.extract()

Pulls the title between the comma and period.

where(..., "Rare")

Groups uncommon titles into one stable category.

fillna("U")

Labels passengers whose deck is unknown.

Create an honest practice test

```
from sklearn.model_selection import train_test_split

train_fe = add_features(train)
X = train_fe.drop(columns="Survived")
y = train_fe["Survived"]

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Why these arguments?

test_size=0.20

Reserves 20% for validation.

random_state=42

Makes the split reproducible.

stratify=y

Keeps a similar survival ratio in both parts.

80% TRAIN

**20%
VALID**

Different columns need different treatment

Numeric

Age · Fare · FamilySize · IsAlone

Categorical

Pclass · Sex · Embarked · Title · Deck

Median imputer

Fills missing numeric values.

StandardScaler

Puts numeric scales on comparable ranges.

Mode + OneHot

Fills and converts categories into indicator columns.



Model-ready matrix

Only numbers, no missing selected values, identical rules everywhere.

Package preparation and model together

```
numeric = ["Age", "Fare", "FamilySize", "IsAlone"]
categorical = ["Pclass", "Sex", "Embarked", "Title", "Deck"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric),
    ("cat", categorical_pipe, categorical)
])
```

Pipeline

Runs ordered steps and learns them during fit.

SimpleImputer

Replaces missing values with learned statistics.

OneHotEncoder

Creates numeric indicators for categories.

handle_unknown="ignore"

Allows a new test category without crashing.

ColumnTransformer

Applies each pipeline to its own columns.

Fit a Logistic Regression baseline

```
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
valid_pred = model.predict(X_valid)
```

What happens inside fit()?

- 1 The imputers learn training medians and modes.
- 2 The encoder learns training categories.
- 3 The scaler learns training means and variation.
- 4 Logistic Regression learns weights that estimate survival.

predict() reuses every learned transformation before producing 0 or 1.

A score is the start of the conversation

```
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    ConfusionMatrixDisplay
)

accuracy = accuracy_score(y_valid, valid_pred)
print(f"Accuracy: {accuracy:.3f}")
print(classification_report(y_valid, valid_pred))

ConfusionMatrixDisplay.from_predictions(
    y_valid, valid_pred
)
```

Accuracy

What fraction of all predictions was correct?

Precision

When we predicted survival, how often were we right?

Recall

Of all survivors, how many did we find?

Ask where the model is wrong

	Predicted 0	Predicted 1
Actual 0	TN Correct non-survivor	FP False survival
Actual 1	FN Missed survivor	TP Correct survivor

Two models can have similar accuracy and make **different kinds of mistakes**.

- ? Which class is harder to predict?
- ? Do errors cluster by sex, class, age, or missing data?
- ? Could a feature or cleaning choice explain the pattern?

Do not tune on the test set: Kaggle keeps its answers hidden. Validation is your local laboratory.

Compare carefully — change one thing

BASELINE

Logistic Regression

- Fast and interpretable
- Linear decision boundary
- Good first reference

NEXT STEP

Decision Tree

- Readable if/then rules
- Captures interactions
- Can overfit quickly

ENSEMBLE

Random Forest

- Combines many trees
- Usually more stable
- Less directly interpretable

Fair comparison: keep the same split, features, preprocessing, and metric. Change only the estimator.

Train on all labels, predict the test set

```
test_fe = add_features(test)

# Refit after choosing the pipeline
model.fit(X, y)
test_pred = model.predict(test_fe)

submission = pd.DataFrame({
    "PassengerId": test["PassengerId"],
    "Survived": test_pred
})

submission.to_csv(
    "/kaggle/working/submission.csv",
    index=False
)
```

Pre-flight checklist

- ✓ Exactly **418** rows.
- ✓ Columns are **PassengerId** and **Survived**.
- ✓ Predictions contain only **0** or **1**.
- ✓ No index column and no missing predictions.

Before upload: run `submission.head()` and `submission.shape`.

Build, evaluate, explain

- 1 Load and re-audit**
Confirm the files, target, and missing values.
- 2 Create four features**
FamilySize, IsAlone, Title, and Deck.
- 3 Build the pipeline**
Impute, scale, encode, then train Logistic Regression.
- 4 Evaluate honestly**
Report accuracy and interpret the confusion matrix.
- 5 Create submission.csv**
Validate its shape, columns, and predictions.

Deliverables

- One runnable Kaggle notebook
- One leakage-safe pipeline
- Validation metrics + confusion matrix
- Three-sentence error analysis
- A valid submission CSV

PART II COMPLETE

A model is a **process**, not one command.

Clean consistently. Engineer meaningful clues. Validate honestly.

QUESTIONS?